

Producto 3.

[Interfaz móvil]

Grupo NoobDevs:

- Kevin Andrés Flores Garces
- Thabata Denise Monteiro
- Pol Valle Montes
- Mar Bou Bernad

Enlace repositorio github

<https://github.com/Des-FrontEnd-con-frameworks-avanz/Producto-3.git>

Enlace trello

<https://trello.com/b/0759UCky/fp451-des-front-end-con-frameworks-avanz-en-ent-mov>

Enlace del proyecto

<https://codesandbox.io/p/github/Des-FrontEnd-con-frameworks-avanz/Producto-3/main>

DOCUMENTACIÓN

Preparación del entorno

Se ha configurado el entorno de desarrollo con las siguientes herramientas

- **Node.js(LTS)**: versión de soporte a largo plazo para asegurar compatibilidad con las dependencias
- **Visual Studio**: IDE principal
- **Android Studio + OpenJDK 21**: imprescindibles para compilar el código de Android y hacer las pruebas en el emulador

El proyecto se inicializó con React Native CLI, que construye el proyecto con el scaffolding nativo usado en react native.

Configuración de Firebase

Como la base de datos fue creada en el Producto 2, en esta fase se ha reutilizado los datos y conexiones del producto anterior.

Se ha usado una arquitectura híbrida heredada de la practica anterior, donde se usa Firebase para la gestión de los datos de la aplicación y dropbox para guardar los vídeos de los jugadores

Para vincular la nueva aplicación React Native a esta base de datos existente se instalaron los módulos oficiales:

npm install @react-native-firebase/app @react-native-firebase/firestore

A nivel nativo Android, se ubicó el archivo de credenciales google-services.json en android/app/ y se habilitó el plugin de Google Services en el build.gradle correspondiente para permitir la compilación.

Listado Screen

Se ha implementado la pantalla ListadoScreen.tsx utilizando TypeScript y componentes de clase. La función principal es recuperar y listar de forma dinámica los perfiles de los jugadores almacenados en Firebase Firestore.

Se utiliza el método getDocs para realizar la petición a la base de datos y un .map para transformar los documentos en un array de objetos tipo Player:

```
async componentDidMount() {
  try {
    console.log('[ListadoScreen] Cargando jugadores desde Firestore...');
    const db = getFirestore();
    const snapshot = await getDocs(collection(db, 'players'));
    console.log('[ListadoScreen] Recibidos ${snapshot.size} jugadores');

    const arrayJugadores: Player[] = snapshot.docs.map((documento) => {
      const datos = documento.data();
      return {
        id: documento.id,
        nombre: datos.nombre,
        apellidos: datos.apellidos,
        posicion: datos.posicion,
        edad: datos.edad,
        altura: datos.altura,
        peso: datos.peso,
        experiencia: datos.experiencia,
        precio: datos.precio,
        fotoUrl: datos.fotoUrl,
        videoUrl: datos.videoUrl,
        posterUrl: datos.posterUrl,
        descripcion: datos.descripcion,
      };
    });

    this.setState({ jugadores: arrayJugadores, cargando: false });
  } catch (error) {
```

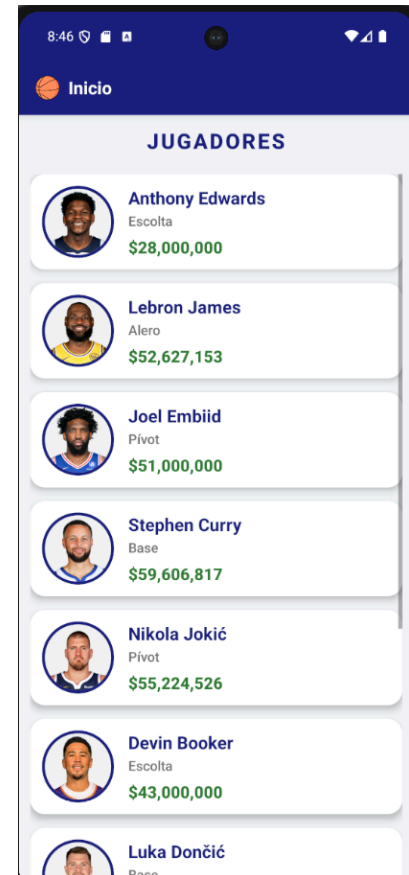
- **FlatList:** Se ha utilizado este componente para optimizar el rendimiento. A diferencia de un mapeo simple, FlatList gestiona eficientemente la memoria al renderizar solo los elementos visibles, lo que garantiza una navegación fluida.
- **TouchableOpacity:** Implementado en cada tarjeta del listado para gestionar la navegación hacia la pantalla de Detalle, enviando el objeto item como parámetro.

```
renderTarjeta = ({ item }: { item: Player }) => {
  return (
    <TouchableOpacity
      style={styles.tarjeta}
      onPress={() => this.props.navigation.navigate('Detalle', { player: item })}
    >
      <Image source={{ uri: item.fotoUrl }} style={styles.foto} />
      <View style={styles.infoContainer}>
        <Text style={styles.nombre}>{item.nombre} {item.apellidos}</Text>
        <Text style={styles.posicion}>{item.posicion}</Text>
        <Text style={styles.precio}>${item.precio.toLocaleString()}</Text>
      </View>
    </TouchableOpacity>
  );
};

render() {
  if (this.state.cargando) {
    return (
      <View style={styles.centro}>
        <ActivityIndicator size="large" color="#1A237E" />
      </View>
    );
  }

  return (
    <View style={styles.contenedor}>
      <Text style={styles.cabecera}>Jugadores</Text>

      <FlatList
        data={this.state.jugadores}
        renderItem={this.renderTarjeta}
        keyExtractor={(item) => item.id as string}
      />
    </View>
  );
}
```



Ventana detalle

La ventana **Detalle** funciona como una **ficha técnica dinámica**, diseñada como un "molde" único que se rellena automáticamente según el jugador seleccionado. Para lograr que la información viaje de una pantalla a otra, utilizamos un sistema de mensajería llamado *route.params*. Al abrir la pantalla, ejecutamos la instrucción `const { player } = route.params;`, lo que nos permite extraer los datos del "paquete" recibido y guardarlos en una constante. Gracias a esto, la pantalla es inteligente: puede mostrar *player.nombre*, *player.posicion* o cualquier otro dato de forma personalizada para cada deportista.

```
const DetalleScreen: React.FC<Props> = ({ route, navigation }) => {
  const { player } = route.params;
  const [verZoom, setVerZoom] = useState(false);
```

Añadimos un botón interactivo mediante el componente **TouchableOpacity**, que actúa como un puente hacia la siguiente sección. Al ser pulsado, ejecuta la función *navigation.navigate*, redirigiendo al usuario a la ventana de **Multimedia**. Lo más importante aquí es que volvemos a pasar el objeto *player* como parámetro, esto garantiza que la aplicación sepa exactamente de qué jugador debe mostrar los videos o jugadas destacadas.

```
<TouchableOpacity
  style={styles.botonMultimedia}
  onPress={() => navigation.navigate('Multimedia', { player })}>
  <Text style={styles.textoBoton}>Ver Mejores Jugadas</Text>
</TouchableOpacity>
```

En cuanto a la estructura visual, organizamos la información dentro de un contenedor principal llamado **infoBox**. En su interior, vamos "mapeando" o asignando cada dato específico a componentes de texto individuales.

```
<View style={styles.infoBox}>
  <Text style={styles.title}>{player.nombre} {player.apellidos}</Text>
  <Text style={styles.filaTexto}>
    <Text style={styles.etiqueta}>Posición: </Text>
    <Text style={styles.subtitle}>{player.posicion}</Text>
  </Text>
</View>
```

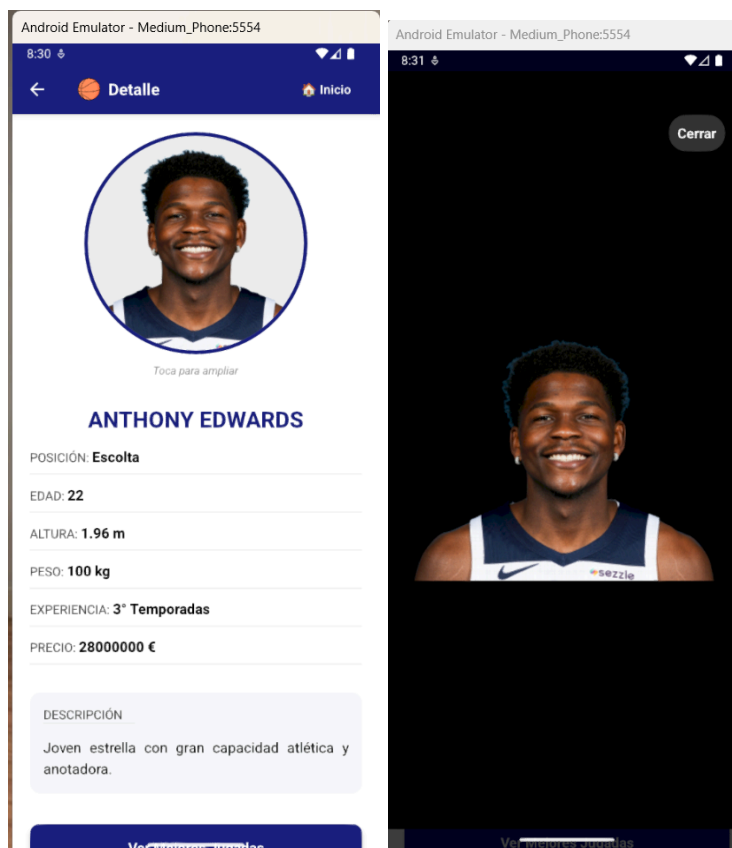
Para la imagen de perfil, implementamos una función de interactividad mediante el evento **onPress**. Al envolver la fotografía en un componente **TouchableOpacity**, permitimos que el usuario pueda hacer clic. Al hacerlo, se dispara la función **setVerZoom(true)**, que cambia el estado de nuestra variable de control.

```
<TouchableOpacity onPress={() => setVerZoom(true)} activeOpacity={0.8}>
  <Image
    source={{ uri: player.fotoUrl }}
    style={styles.imagenRedonda}
  />
</TouchableOpacity>
```

Para gestionar la visualización ampliada, utilizamos un concepto fundamental en React llamado **Estado**. Definimos una constante booleana, `const [verZoom, setVerZoom] = useState(false)`, que funciona como un interruptor: inicialmente está apagado (*false*). Cuando este valor cambia

a verdadero, se activa el componente **Modal**. Este modal se configura como una capa superior que cubre toda la pantalla con un fondo oscuro, donde mostramos la imagen nuevamente pero con el estilo *imagenZoom* y la propiedad *resizeMode="contain"*. Esto asegura que la foto se vea en grande y completa sin deformarse.

```
<Modal visible={verZoom} transparent={false}>
  <View style={styles.modelFull}>
    <TouchableOpacity style={styles.botonCerrar} onPress={() =>
setVerZoom(false)}>
      <Text style={styles.textoBoton}>Cerrar</Text>
    </TouchableOpacity>
    <Image
      source={{uri: player.fotoUrl}}
      style={styles.imagenZoom}
      resizeMode="contain"
    />
  </View>
</Modal>
```



Multimedia Screen

Se instala react-native-video, esto nos permite usar el componente VideoView para poder crear el reproductor multimedia.

Implementamos un hook para obtener la url de firebase y iniciar la reproducción con play().

```
const videoPlayer = useVideoPlayer(  
  { uri: player.videoUrl ?? '' },  
  (p) => {  
    p.play();  
  },  
);
```

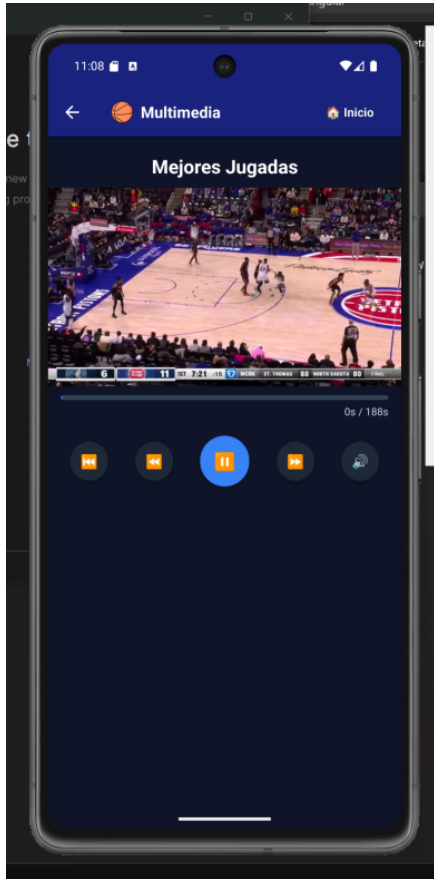
Mediante useState se controlan los estados del reproductor, paused, muted, currentTime y duration.

Se utiliza useEffect para actualizar el progreso en tiempo real, obtener la duración del video y detectar la finalización.

Para cumplir la rúbrica se implementan los siguientes botones, reiniciar, retroceder 10segundos, play/pausa, avanzar 10segundos y mute/unmute el áudio. Además se implemente una barra visual del estado del video.

Finalmente se integra en el componente DetalleScreen, en este se pasa el objeto player.

```
<TouchableOpacity  
  style={styles.botonMultimedia}  
  onPress={() => navigation.navigate('Multimedia', { player })}>  
  <Text style={styles.textoBoton}>Ver Mejores Jugadas</Text>  
</TouchableOpacity>
```



Navegación entre ventanas

Para la navegación se han instalado las librerías oficiales de React Navigation v7:

```
npm install @react-navigation/native @react-navigation/native-stack react-native-screens react-native-safe-area-context
```

Se ha optado por native-stack en lugar del stack JS clásico porque delega las transiciones al sistema operativo (mejor rendimiento) y aporta de forma automática la flecha de retroceso que exige la rúbrica.

Bibliografía

React Native Documentation: <https://reactnative.dev/docs/flatlist>

Firebase for React Native: <https://rnfirebase.io/firestore/usage>